

TOS Network

Open Coordination and Settlement for AI Services and Physical Edge Intelligence

TOS Network

2026 Edition

Abstract

TOS Network is being built as an open coordination and settlement network for autonomous agents, owner-operated AI services and site-bound physical AI terminals. Its economic unit is a completed service action under explicit policy, not an hour of unidentified hardware. Providers retain control of their models, devices, sites, data and operational responsibility; the network is intended to standardize identity, discovery, authorization, service commitments, evidence and settlement across organizational boundaries.

The implemented foundation, TOS Core, is an actor-model blockchain with native TVM execution, value-bearing asynchronous messages, masterchain-rooted consensus, dynamic sharding and ADNL/DHT/RLDP networking. It includes foundations for Agent Accounts, concurrent Service Actors, Task Escrow, Dispute, Capability Registry and Proof Attestation. The broader service protocol, public discovery products and AI Edge Computing Terminals described in this paper are planned product layers and are not presented as already deployed commercial infrastructure. Models, prompts, sensor streams and private customer data remain off-chain; shared authority, commitments and settlement use the chain where independently operated parties require a tamper-resistant source of truth.

Contents

1	Executive Summary	4
1.1	Strategic Positioning	4
2	Design Principles	4
2.1	Service Outcomes, Not Bare Capacity	4
2.2	Actor First	5
2.3	Asynchronous by Default	5
2.4	Authority Must Be Explicit	5
2.5	Verification Without Payload Bloat	5
2.6	Local Control and Safety	5
2.7	Bounded Resources and Recoverable Failure	5
2.8	Native Execution Focus	5
3	System Architecture	6
3.1	Layer and Repository Boundaries	6
3.2	Masterchain and Basechain	6
3.3	Accounts as Actors	6

3.4	Cells and TVM	7
3.5	Asynchronous Messages	7
3.6	Sharding	7
3.7	Consensus and Networking	7
4	Trust and Settlement Actors	7
4.1	Agent Wallet	7
4.2	Agent Account	8
4.3	Task Actor	8
4.4	Capability Registry Actor	8
4.5	Service Actor	8
4.6	Verifier and Attestation Actors	9
5	Agent Wallet and Authority Model	9
5.1	Owner and Controller Separation	9
5.2	Operational Lifecycle	9
5.3	Policy and Permission Linkage	9
6	Task Markets and Settlement	9
6.1	Creating Work	9
6.2	Assignment and Claim	10
6.3	Results and Evidence	10
6.4	Settlement and Disputes	10
7	TOS Service Protocol and Discovery	10
7.1	Base Protocol	10
7.2	Capability Discovery	10
7.3	Names and Reachability	10
8	AI Edge Computing Terminals	11
8.1	Terminal as the Product Unit	11
8.2	Runtime and Executor Isolation	11
8.3	Admission, Scheduling and Resource Bounds	11
8.4	Site-Bound Physical AI	11
8.5	Disconnected Operation and Reconciliation	12
8.6	Safe Model and Software Updates	12
8.7	Fleet Management	12
9	Verifiable AI Workflows	12
9.1	State and Evidence Boundary	12
9.2	Proof Adapters	13
9.3	Managed-Service Workflow	13
9.4	Physical-Terminal Workflow	13
10	APIs and Operator Tooling	13
10.1	tosctl	13
10.2	Read APIs	13
10.3	Trust Tiers	14

11 Security Model	14
11.1 Key Compromise	14
11.2 Replay and Domain Confusion	14
11.3 Escrow Safety	14
11.4 Service and Evidence Risk	14
11.5 Executor and Supply-Chain Isolation	14
11.6 Physical Safety	15
11.7 Offline Authority and Update Risk	15
11.8 Fleet Compromise	15
11.9 Indexer Authority Drift	15
11.10 Availability and Message Amplification	15
12 Economics	15
12.1 Native Unit	15
12.2 Initial Supply	15
12.3 Service Economic Flows	16
13 Implementation Status	16
14 Verification and Release Gates	17
15 Roadmap	18
15.1 Foundation: TOS Core	18
15.2 Phase 0: Base Protocol and Edge Core	18
15.3 Phase 1: Managed Inference Terminal	18
15.4 Phase 2: Site-Bound Physical Terminal	18
15.5 Phase 3: Additional Service Profiles	18
15.6 Phase 4: Advanced Network Services	18
16 Non-Goals	18
17 Conclusion	19

1 Executive Summary

AI is moving from centralized applications toward autonomous software and independently operated model services. At the same time, inference is moving toward cameras, robots, vehicles, factories, stores and homes where latency, privacy, intermittent connectivity and physical safety matter. This creates a fragmented economy of agents, models, devices, data, tools and human services. The resources can perform useful work, but they do not share a neutral system for persistent identity, machine-readable capability, bounded authorization, payment, receipts, evidence and dispute resolution.

TOS Network is designed to provide that common trust and economic layer. It does not compete to train foundation models, operate a centralized cloud or rent bare accelerators. At the application-network layer, it follows an asset-light coordination model: hardware, models, sites and operations remain with providers, while protocol participants standardize how services are described, invoked, evidenced and settled.

The architecture has five layers:

1. **TOS Core.** Native actor execution, consensus, sharding, identity primitives, payment and authenticated networking.
2. **TOS Service Protocol.** Versioned descriptors, authentication, quotes, payment authorization, receipts, evidence envelopes and profile negotiation.
3. **Owner-operated terminals.** Bounded local execution through approved service adapters without exposing a public shell, container socket or raw accelerator.
4. **Discovery and clients.** Human-readable names, capability search, policy-aware selection and verifiable resolution.
5. **Vertical profiles.** Managed AI, physical AI, storage, commerce and human services that share common protocol foundations but retain distinct state machines.

Only the first layer is substantially implemented in this repository. The remaining layers are a product architecture and delivery plan. This distinction is deliberate: validators do not execute application workloads, and a new model, accelerator or service profile must not require a consensus upgrade.

1.1 Strategic Positioning

The unit of value in TOS Network is a completed, policy-compliant service action. A consumer requests a capability with an exact service revision, input and output contract, price, deadline, region, data-egress policy and evidence level. An owner-operated terminal decides whether it can safely admit the request, executes it through an approved local runtime, returns a signed receipt and settles according to the agreed payment policy.

For physical terminals, the value is often location rather than raw throughput. A low-power terminal beside a production line, camera or robot can provide low-latency and privacy-preserving execution that a remote cloud cannot provide economically or safely. Local safety and deterministic real-time work always take priority, and TOS never replaces an independent safety controller.

2 Design Principles

2.1 Service Outcomes, Not Bare Capacity

Consumers buy a versioned service capability and a declared outcome. Hardware declarations, TOPS, accelerator memory and benchmarks are scheduling evidence; they are not themselves the

service contract. Bare GPU rental, public accelerator access and arbitrary consumer-supplied programs or containers are outside the TOS product plan.

2.2 Actor First

Every account is an actor. Agent Accounts, Task Escrows, capability entries, service actors and verifier actors are ordinary native accounts with isolated state. There is no privileged global application process.

2.3 Asynchronous by Default

Cross-actor workflows use messages, callbacks, deadlines and retries. A sender cannot assume that another actor executes synchronously or that a multi-step workflow is atomic. Contracts must expose explicit intermediate states and deterministic failure behavior.

2.4 Authority Must Be Explicit

Owner, controller, creator, assigned agent, verifier, service and fee-payer roles are distinct. Every transfer or state transition must identify its authority. Off-chain runtimes are not trusted merely because they operate an agent.

2.5 Verification Without Payload Bloat

The chain is authoritative for funds, permissions, deadlines and settlement. Large prompts, model outputs, datasets and private credentials remain off-chain. Contracts store hashes, signatures, attestations and evidence references that bind external work to on-chain state.

2.6 Local Control and Safety

Providers retain custody of hardware, data, models and operational policy. A physical terminal continues approved local work while disconnected and keeps blockchain consensus outside the hard real-time and actuator-control loop. Raw sensor and physical-I/O interfaces are not public network capabilities.

2.7 Bounded Resources and Recoverable Failure

Connections, queues, retries, model caches, RAM, accelerator memory, durable journals and watchers must have explicit limits and cleanup paths. Failure, cancellation, restart, disconnect and payment rejection are normal protocol states. No anonymous or unpaid caller may cause unbounded memory, disk or durable-state growth.

2.8 Native Execution Focus

TOS Core has one active application execution domain: the native TVM basechain. Unsupported execution engines are not registered in the current product. Application workloads run in separate terminal processes and repositories, not inside `validator-engine`.

3 System Architecture

3.1 Layer and Repository Boundaries

TOS Network is designed as independently released layers. The separation limits consensus risk and allows product profiles to evolve at application speed.

Layer	Responsibility	Delivery boundary
TOS Core	Consensus, TVM, generic contracts, query APIs, wallet and cryptography, DNS, ADNL/DHT/RLDP and TOS Sites	Implemented and maintained in the <code>tos</code> repository
Base service plane	Descriptors, manifests, authentication, quotes, payment authorization, receipts, evidence, discovery schema, Edge Core and conformance	Planned separate <code>tos-protocol</code> repository
AI terminals	General and physical terminal products, resource probes, runtime adapters, scheduling, signed updates and fleet management	Planned separate <code>tos-ai</code> repository
Other profiles	Storage leases, commerce, fulfillment and human-service state machines	Planned independently released vertical repositories

Existing Agent, Service, Escrow, Registry and Attestation contracts remain reusable TOS Core foundations. Product-specific daemons, model runtimes, catalogs and continuous device telemetry do not belong in the validator repository.

3.2 Masterchain and Basechain

The masterchain anchors validator sets, consensus configuration, workchain descriptors and cross-shard finality. The basechain executes native TVM accounts and contracts. The canonical genesis registers the masterchain with identifier -1 and the native basechain with identifier 0 .

The masterchain does not execute an AI model. It establishes the shared state required for agents and task actors to trust message delivery, account state and settlement outcomes.

3.3 Accounts as Actors

An account transition can be represented as

$$(S_{t+1}, M_{out}) = F(S_t, M_{in}, C_t),$$

where S_t is the account state, M_{in} is one inbound message, C_t is the consensus execution context and M_{out} is a finite set of outbound messages. The transition changes only the receiving account. Effects on another account occur later when an emitted message is delivered there.

This isolation maps directly to AI systems: planners, workers, tools, tasks and verifiers can progress independently while retaining a shared, auditable settlement layer.

3.4 Cells and TVM

TOS stores account code and persistent state in immutable cells. Cells form directed acyclic graphs and are addressed by cryptographic hashes. TVM executes contract code deterministically, charges gas and emits messages. Compact cell-native state is well suited to balances, policy, status values and content commitments.

3.5 Asynchronous Messages

Messages carry a destination, optional source, value and body. Internal messages have an unforgeable source account. External messages may carry signatures and replay-protection data. Value transfer is therefore part of the same mechanism as task acceptance, service calls and settlement.

AI workflow messages should include an opcode, a correlation value such as `query_id`, clear value semantics and enough domain binding to prevent replay against another account, task or network.

3.6 Sharding

TOS follows a bottom-up sharding model. Logically, each account has an independent history; physically, many account histories are grouped into shardchains. Shards may split and merge as load changes. Cross-shard messages preserve the actor abstraction even when communicating accounts reside in different shards.

This model permits parallel execution because transactions affecting different destination accounts do not mutate shared application state. High-volume agent economies can distribute tasks and services across many accounts instead of concentrating all workflow state in one contract.

3.7 Consensus and Networking

Validators agree on masterchain and shardchain blocks and enforce protocol configuration. The networking stack provides authenticated datagram transport, distributed discovery, reliable large-message delivery and overlay communication. ADNL identities and TOS DNS can locate services; RLDP and TOS Sites can carry application traffic. These primitives do not by themselves provide the planned service manifest, quote, receipt, relay or terminal product. `validator-engine` remains the canonical node process; `tosctl` provides an operator path for configuration and existing agent-oriented workflows.

4 Trust and Settlement Actors

4.1 Agent Wallet

An Agent Wallet is the custody and local-profile layer for an autonomous agent. The local profile records the underlying native wallet, owner key reference, controller key reference, spending policy, capabilities, optional metadata commitments and optional runtime binding.

Creating a profile generates separate owner and controller keys in the configured secrets vault. Funding transfers TOS to the derived native wallet address. Activation deploys the underlying wallet contract after the address has sufficient balance.

The owner retains full custody authority. Manual owner transfers and emergency operations are operator actions. Automated agent spending must use the policy-enforced Agent Account path. The owner key must never be exported to an off-chain agent runtime.

4.2 Agent Account

An Agent Account is the on-chain authority boundary for automated actions. Its state includes:

- owner address and controller public key;
- sequence number and expiry checks for replay protection;
- maximum value per controller action;
- cumulative daily limit and current daily spend;
- default task timeout;
- optional metadata and service-endpoint hashes.

Owner-authorized internal messages update policy or rotate the controller. Controller-signed external messages may send bounded task actions only after signature, expiry, sequence, per-action and UTC-day checks succeed. Local CLI checks improve operator feedback, but the contract is the final authorization boundary.

4.3 Task Actor

A Task Escrow is an account representing one unit of work. It stores the creator, optional assigned agent, optional verifier, budget, deadline, review period, lifecycle status, result hash, evidence hash, settlement-policy hash, permission hash and dispute hash.

The implemented lifecycle includes:

Open -> Accepted -> Submitted -> Settled

with terminal or review paths for rejection, cancellation, expiry and dispute resolution. An unassigned open task may be claimed atomically; the first valid claim records the caller as the agent. Result submission starts a review window. Authorized settlement releases a bounded payout and returns any remainder according to contract rules.

4.4 Capability Registry Actor

A Capability Registry entry is one contract per registered agent or service. It is not a chain-wide dictionary and does not by itself provide complete network enumeration. Its state commits to:

- owner and optional verifier;
- active or inactive status and registration time;
- task-category, pricing, metadata and verification-method hashes;
- a stakeable bond;
- an accumulating signed reputation score.

Owner-authorized operations update metadata, rotate the verifier, withdraw bond, deactivate or reactivate the entry. Staking is permissionless. Reputation updates require verifier authority. The registry exposes inspectable commitments while allowing detailed service descriptions to remain off-chain.

4.5 Service Actor

A Service Actor represents a bounded model, data, tool or other defined service. The implemented contract supports open access or one authorized caller, price per call, a UTC-day rate limit, metadata and proof-scheme hashes, request and response commitments, accumulated revenue and an active or inactive lifecycle. A call must attach at least the configured price; accepted value is credited to contract revenue. The owner commits response hashes, updates policy, withdraws revenue and

controls activation. The registry says what a service claims to provide; the Service Actor governs calls and payments. It does not expose a host, raw accelerator or arbitrary execution environment.

4.6 Verifier and Attestation Actors

A Verifier Actor evaluates result or evidence commitments according to a declared policy. A verifier may accept, reject, score or dispute work, but its authority must be explicitly stored by the task or registry actor. The implemented Proof Attestation foundation can record compact claims and evidence references. Neither a payment nor a hash alone proves semantic correctness, confidentiality or the physical hardware used. TOS does not hard-code one model, oracle or proof backend.

5 Agent Wallet and Authority Model

5.1 Owner and Controller Separation

The owner controls custody, recovery and high-authority policy changes. The controller is an automation key with bounded authority. Compromise of a controller should not become owner-equivalent compromise.

For a controller action with value v , per-action limit L_a , daily limit L_d and already spent amount D , the Agent Account requires

$$0 < v \leq L_a \quad \text{and} \quad D + v \leq L_d.$$

It also verifies a valid signature, an expected sequence number and a non-expired timestamp.

5.2 Operational Lifecycle

The initial operator lifecycle is:

1. create a local Agent Wallet profile and vault keys;
2. fund and activate the underlying native wallet;
3. build and inspect deterministic Agent Account state;
4. deploy the Agent Account through an active funding wallet;
5. bind an off-chain runtime to the profile;
6. execute bounded task actions with the controller;
7. use the owner for policy updates, controller rotation and explicit custody actions.

5.3 Policy and Permission Linkage

Task actions can bind a locally tracked permission identifier to an on-chain permission hash. Before signing, tooling checks the target contract, lifecycle state, assignment and permission linkage. The Task Escrow independently validates sender authority and state transition rules. This layered design catches operator mistakes without treating local configuration as chain authority.

6 Task Markets and Settlement

6.1 Creating Work

A creator deploys a Task Escrow with a budget, deadline, settlement policy and optional agent or verifier. The deployment value must cover the intended escrow and execution costs. Task metadata itself may be stored off-chain; the contract retains compact commitments.

6.2 Assignment and Claim

A task can target a specific Agent Account or remain open. A targeted agent may accept or reject according to contract state. An open task may be claimed atomically. Assignment is authoritative only after it appears in chain state.

6.3 Results and Evidence

The agent submits a result hash and evidence hash. These values do not prove semantic quality by themselves; they bind a later-retrieved artifact or attestation to the task. The review period gives the creator or verifier time to settle or dispute the submission.

6.4 Settlement and Disputes

Settlement transfers no more than the available escrow. Cancellation and rejection refund according to explicit lifecycle rules. Timeout is consensus-time based. A dispute records a reason or evidence commitment and moves authority to the configured resolution path. Verifier resolution is bounded by contract balance and role checks.

7 TOS Service Protocol and Discovery

7.1 Base Protocol

The planned TOS Service Protocol is the common application plane above TOS Core. Independent implementations require canonical and versioned definitions for:

- signed service descriptors and short-lived terminal manifests;
- profile negotiation and handling of unknown critical extensions;
- authentication, replay domains, nonces and bounded delegation;
- quotes, payment authorization, cancellation, refunds and idempotency;
- request, stream, result, receipt and evidence envelopes;
- durable operation identities and restart recovery;
- error classes, limits and compatibility rules.

This protocol is planned work, not functionality implied by the existence of a Service Actor contract. The contract is a settlement primitive; an interoperable session and receipt protocol still requires schemas, SDKs, conformance vectors and independent implementations.

7.2 Capability Discovery

Capability discovery separates compact on-chain commitments from extensible off-chain descriptions. A client can inspect an entry's owner, activity, bond and commitment hashes, then retrieve a signed descriptor and verify its relationship to chain state.

The current per-actor registry does not solve chain-wide search. Complete discovery requires an indexer or protocol registry that can enumerate and rank services. Indexed data is a derived view: clients must verify critical owner, authorization, capability, activity and payment fields against signed or on-chain state before committing funds.

7.3 Names and Reachability

A service may be reached through a human-readable `name.tos` identity or a raw ADNL identity. A descriptor may publish RLDP/TOS Sites and optional HTTPS, QUIC, WebSocket or streaming

bindings without changing the identity and authorization model. Ordinary home and site networks may require an owner-selected relay or reverse tunnel. A relay may forward encrypted traffic but must not receive owner authority, unrestricted runtime credentials or settlement control.

A public root `.tos` registration product, signed multi-endpoint descriptors, short-lived health data and production relay service are planned. DNS and TOS Sites primitives already exist, but they are not equivalent to a complete service discovery marketplace.

8 AI Edge Computing Terminals

8.1 Terminal as the Product Unit

The TOS AI Edge Computing Terminal is the first planned owner-operated execution product. It combines measured local resources, approved runtime adapters, bounded task admission, TOS identity and reachability, metering and signed receipts. A terminal may be a Linux GPU workstation, an AI PC, a small edge server or an industrial ARM/Jetson appliance.

Consumers schedule against a service capability such as text generation, embedding, OCR, speech recognition or video analysis. A capability states the exact model or task revision, schemas, limits, privacy and region policy, evidence level, price and expiry. Hardware model numbers and vendor performance claims are evidence used by the scheduler; the consumer does not receive raw access to the device.

8.2 Runtime and Executor Isolation

The public service listener is separated from the administrative interface. Public adapters run under dedicated operating-system identities, containers or equivalent sandboxes with explicit CPU, RAM, accelerator-memory, filesystem, network and tool grants. They receive no Docker socket, shell, owner key, unrestricted wallet key, home directory or raw physical-I/O authority.

The operator approves every model and executable artifact in the initial product. A runtime adapter performs compatibility checks, bounded load and unload, cancellation, error normalization, usage reporting and cleanup after success, timeout, disconnect, out-of-memory failure or process crash. Consumer-supplied containers, native programs and models are not accepted by the TOS terminal product.

8.3 Admission, Scheduling and Resource Bounds

The local scheduler is authoritative for admission. It considers model compatibility, RAM, accelerator memory, storage, queue and in-flight limits, context and output bounds, deadlines, owner reservations, thermal policy, price, privacy and evidence requirements. Remote discovery cannot reserve local hardware merely by publishing an optimistic capacity value.

Every implementation must bound connections, sessions, streams, nonces, quotes, queues, retries, model caches, temporary files, logs, receipt queues, chain watchers and durable journals. Completion, cancellation, failed payment, adapter crash, upgrade and shutdown each need a tested cleanup path. These constraints are required to prevent anonymous RAM, VRAM, disk, watcher and durable-state growth.

8.4 Site-Bound Physical AI

A physical terminal runs beside sensors, robots, vehicles or industrial equipment. Its mandatory priority order is:

1. emergency and independent safety interlocks;
2. deterministic control deadlines;
3. local real-time perception and sensor fusion;
4. local asynchronous analysis and maintenance;
5. approved external service requests;
6. background update, telemetry and compaction.

TOS does not enter the hard real-time loop. Raw CAN, GPIO, serial, fieldbus, camera or actuator interfaces are never public capabilities. A narrow semantic action must pass local policy and an independent safety controller, which may reject an otherwise valid network request.

8.5 Disconnected Operation and Reconciliation

A physical terminal must continue its approved local workload while disconnected. It may use only unexpired cached policy and explicitly bounded offline authority. Events and obligations are written to a size- and age-bounded, tamper-evident journal. Reconnection first observes revocation and policy expiry, then uploads and reconciles events idempotently. A reconnect must not execute a paid task twice, lose a refund obligation or grow the journal without limit.

8.6 Safe Model and Software Updates

Model, runtime, firmware and policy updates are content-addressed, signed by an authorized release identity, compatibility-checked and staged before activation. Terminals maintain an active slot and a known-good rollback slot. Activation is crash-safe and subject to health gates, anti-rollback policy and bounded artifact retention. Fleet rollouts use canary and progressive rings and stop automatically when declared health thresholds fail.

8.7 Fleet Management

Fleet state is primarily off-chain. A fleet service handles enrollment, scoped delegation, site grouping, staged rollout, health, revocation, offline expiry and retirement without publishing detailed topology or continuous telemetry on-chain. Commands have bounded fan-out, expiry and idempotency. Compromise of a fleet controller must not confer owner custody or override independent local safety policy.

9 Verifiable AI Workflows

9.1 State and Evidence Boundary

TOS records what consensus can enforce:

- account authority and spending limits;
- task assignment, status, budget and deadlines;
- settlement and dispute authority;
- result, evidence and metadata commitments;
- registry bond, activity and reputation references.

The chain should not store private prompts, service credentials, full model transcripts or large datasets. These remain in content-addressed storage or service infrastructure.

9.2 Proof Adapters

Verification may use signatures, attestations, proof-backed transport evidence, committee decisions or domain-specific proof systems. A proof adapter translates external evidence into a compact decision or commitment understood by a task or verifier actor. The core protocol remains neutral to the model provider and proof backend.

Claims must state their evidence level. A terminal declaration, an observed successful call, a standardized benchmark, an independent audit, hardware attestation and replicated execution are different forms of evidence and must not be presented as interchangeable. A signed receipt proves that a signer made a statement about an operation; its semantic meaning still depends on the accepted policy and trust model.

9.3 Managed-Service Workflow

1. A client resolves a service identity and verifies its signed descriptor.
2. The client requests a quote for an exact profile, limits and evidence policy.
3. A controller or user authorizes a bounded payment against the signed quote.
4. The terminal admits the request only after local policy and resource checks.
5. An isolated adapter executes the approved model and streams a bounded response.
6. The terminal signs a result and usage receipt linked to the operation identity.
7. The client verifies the receipt and the settlement path releases or refunds value.

Every step is asynchronous. Failure, timeout and dispute are explicit workflow states rather than hidden exceptions in an off-chain orchestration process.

9.4 Physical-Terminal Workflow

A site terminal receives sensor data locally, schedules safety and real-time perception ahead of all network work, applies local policy and emits a bounded event or service result. If the site is disconnected, it continues approved local operation and journals the event. After reconnection it observes revocations, reconciles the journal idempotently and publishes only the receipt or evidence allowed by site data policy. Blockchain availability is therefore not a prerequisite for an emergency stop or deterministic local control decision.

10 APIs and Operator Tooling

10.1 `tosctl`

`tosctl` is the canonical operator tool for node configuration, native wallets and AI actor workflows. The implemented command families create and inspect Agent Wallet profiles, deploy and manage Agent Accounts, create and operate Task Escrows, and deploy or update Capability Registry entries.

Human-readable output supports operators; JSON output supports agent runners and automation. Secrets remain in the configured vault rather than the profile JSON.

10.2 Read APIs

The authenticated node-control service exposes read endpoints without URI version prefixes:

- `GET /agents/{address};`
- `GET /tasks/{address};`
- `GET /tasks` with status, creator, agent, deadline and pagination filters.

Task-list chain reads use bounded concurrency, deterministic ordering, individual timeouts and failure isolation. The list enumerates tasks registered in the local node-control configuration; it is not a complete chain-wide index.

Public API errors distinguish invalid requests, missing state, unavailable RPC, invalid contract state and timeout without exposing raw internal transport errors.

10.3 Trust Tiers

An agent controlling material funds should verify state through its own node or a proof-backed client. A trusted remote API may be suitable for an operator's own infrastructure. Third-party indexers are useful for search and analytics but must not become authority for balances, permissions or settlement.

11 Security Model

11.1 Key Compromise

Owner keys belong in operator-controlled vaults. Controller keys belong to runtimes only when their on-chain authority is acceptably bounded. Rotation must be possible without changing the owner. Runtime manifests must never contain owner secrets.

11.2 Replay and Domain Confusion

Signed actions require sequence numbers and expiry. Protocols should bind signatures and permissions to the intended network, account, task and operation. A valid message for one workflow must not authorize another.

11.3 Escrow Safety

Contracts reject out-of-order messages, unauthorized senders and payouts above available balance. Deadlines, review windows and terminal states are explicit. Local CLI validation is not a substitute for contract checks.

11.4 Service and Evidence Risk

A capability claim is not proof of service quality. A metadata hash is not proof that its content is true. A result hash is not proof that a result is correct. Bonds, verifier roles, attestations and reputation can reduce risk, but task settlement policy must state which evidence is authoritative.

11.5 Executor and Supply-Chain Isolation

Public requests are untrusted input. Runtime adapters must enforce schema, byte, dimension, duration and output limits before allocating expensive resources. Models, runtimes and updates require authenticated provenance, canonical hashes, compatibility checks and operator approval. Sandboxing, least-privilege device access, network egress policy and secret separation limit the impact of a compromised model server or parser. The public service interface must never become an administrative or arbitrary-code execution path.

11.6 Physical Safety

Possession of a valid TOS key, payment authorization or network receipt does not grant raw actuator authority. Local semantic capability policy and independent safety interlocks remain authoritative. Safety and control deadlines pre-empt external work; overload or network failure must fail toward a locally defined safe state.

11.7 Offline Authority and Update Risk

Offline operation creates stale-policy and replay risk. Offline rights therefore need explicit scope, expiry and budgets, while journals need integrity, bounded retention and idempotent reconciliation. Update systems require separate release authority, anti-rollback checks, known-good recovery and staged fleet rollout. A terminal that cannot verify a package or recover a healthy runtime must continue only its previously approved safe workload.

11.8 Fleet Compromise

Fleet controllers use scoped delegation and revocable terminal identities. Commands carry target scope, expiry and idempotency identifiers. Rollout fan-out, retry queues, health history and offline inventory are bounded. Fleet compromise must not reveal owner custody keys, customer payloads or unrestricted site topology.

11.9 Indexer Authority Drift

Search results and reconstructed timelines are non-authoritative. Critical clients should re-read relevant contract state before transferring funds, accepting work or resolving a dispute.

11.10 Availability and Message Amplification

Automated actors can create unbounded retry loops. Production workflows require funded and bounded retries, backoff, timeout policy, queue limits and observable failure records. Terminals additionally require soak tests for RAM, accelerator memory, disk, watchers, connections, offline journals and update artifacts across success and every failure path.

12 Economics

12.1 Native Unit

TOS is the native settlement asset. One TOS equals 10^9 *nanotomi*, the smallest indivisible unit. Contract balances, fees, task budgets, bonds and spending limits are represented in *nanotomi* to avoid rounding ambiguity.

12.2 Initial Supply

The canonical genesis target supply is 5,000,000,000 TOS. It is allocated at zero-state construction time, primarily to the main wallet with operational balances assigned to the elector and configuration contracts. Native genesis does not register proof-of-work givers.

12.3 Service Economic Flows

The architecture can use native TOS for:

- escrowed task budgets and agent payouts;
- refunds after rejection, cancellation or timeout;
- service-call settlement under explicit quotes and maximum charges;
- registry bonds and future verifier staking;
- defined discovery, relay, verification or storage services;
- gas for messages and persistent actor state.

Economic activity is designed to arise from completed service transactions and explicitly defined infrastructure services, not from merely remaining online or declaring hardware capacity. Token ownership does not grant an agent unlimited automation authority, prove service quality or override terminal policy.

Service transaction volume, validator fees, protocol revenue and token value accrual are distinct concepts. This paper makes no forecast or promise regarding any of them. Detailed monetary policy, incentives, fee routing and governance require separate specifications and accepted network configuration.

13 Implementation Status

This whitepaper separates repository implementation from architectural direction. “Implemented” means this repository contains the relevant node, contract, tooling or test foundation. It does not imply public commercial deployment, external audit, production service availability or adoption. “Partial” means a reusable primitive exists but the interoperable product surface does not. “Planned” identifies work assigned to a future protocol or product repository.

Capability	Status	Scope
Native TVM execution and actor messages	Implemented	Node, contracts, cells, gas and value-bearing messages
Masterchain and shardchain protocol	Implemented	Validator and full-node protocol stack
ADNL, DHT, RLDP, QUIC and TOS Sites	Implemented	Core transport and networking primitives
TOS DNS and resolver chaining	Implemented	Naming primitives; not a complete public registrar product
Agent Wallet profiles	Implemented	Keys, policy, funding, activation, runtime binding and inspection
Agent Account	Implemented	Deterministic deployment, controller actions, limits and rotation
Task Escrow	Implemented	Claim, accept, reject, result, review, dispute, resolve and settlement
Agent and task read APIs	Implemented	Authenticated reads, filtering, timeout and error taxonomy
Capability Registry	Implemented	Per-actor entry, metadata hashes, bond, activity and reputation
Service Actor contract	Implemented	Access policy, pricing, rate limits, calls, responses and revenue

Capability	Status	Scope
Dispute and Proof Attestation foundations	Implemented	Compact resolution and evidence commitments
Public .tos registrar product	Planned	Ownership lifecycle, SDK, deployment and operator tooling
TOS Service Protocol	Planned	Descriptors, sessions, quotes, receipts, evidence and conformance
Edge Core and discovery clients	Planned	Terminal enforcement, SDKs, search and verifiable resolution
Managed AI terminal	Planned	Approved model services, bounded scheduling, metering and receipts
Physical AI terminal	Planned	Offline operation, safe updates, real-time priority, isolation and fleet management
Chain-wide service index	Planned	Search without treating indexed data as authority
Public-testnet hardening and external audit	Planned	Persistent deployment, load testing and independent review

14 Verification and Release Gates

No terminal profile is production-ready merely because its happy path works. Release suites must cover:

- canonical encoding, signature domains, replay, expiry, idempotency and unknown critical extensions;
- contract authorization, escrow conservation, cancellation, refunds and disputes;
- resolver, descriptor, quote, payment, invocation, receipt and restart flows on a multi-node TOS network;
- adapter crash, timeout, disconnect, malformed input, out-of-memory and failed-payment cleanup;
- bounded RAM, accelerator memory, disk, cache, watcher, connection, retry and journal behavior under sustained load;
- physical real-time priority, network partition, stale policy, reconnect and idempotent reconciliation;
- signed update verification, compatibility rejection, power-loss activation, rollback and staged fleet abort;
- actuator isolation, local safety override, fleet revocation and bounded command fan-out.

Release gates include a threat model, protocol conformance vectors, reproducible builds, system-service or container hardening, migration and rollback procedures, independent review of externally exposed and settlement-critical components, and observed testnet operation.

15 Roadmap

15.1 Foundation: TOS Core

Maintain consensus, native TVM execution, networking, wallet and reusable actor-contract foundations as a stable dependency. Application workloads and vertical product release cycles remain outside the validator.

15.2 Phase 0: Base Protocol and Edge Core

Specify canonical descriptors, manifests, authentication, quotes, payment authorization, receipts, evidence levels, profile negotiation and conformance vectors. Build the public `.tos` registrar, chain adapter, SDKs, Edge Core, discovery schema and a deterministic multi-node reference harness.

15.3 Phase 1: Managed Inference Terminal

Deliver a Tier 1 Linux/NVIDIA reference terminal for provider-approved models and bounded AI services. Complete resource probes, initial runtime adapters, task admission, streaming, metering, signed receipts, restart recovery and resource-soak testing. This phase does not accept arbitrary consumer execution.

15.4 Phase 2: Site-Bound Physical Terminal

Deliver a Jetson/ARM reference profile with disconnected local execution, bounded offline authority, tamper-evident reconciliation, signed A/B updates, real-time priority, physical-I/O isolation, independent safety interlocks and staged fleet management.

15.5 Phase 3: Additional Service Profiles

Add storage, commerce, digital and physical fulfillment, human-service and controlled composition profiles without forcing their business state machines into the AI protocol or validator.

15.6 Phase 4: Advanced Network Services

Add production relays, prepaid or channel settlement, multi-region routing, replication, stronger attestation, verifier markets and policy-aware orchestration after the base protocol and terminal cleanup invariants are stable.

16 Non-Goals

TOS does not aim to:

- run private model inference directly in consensus;
- store prompts, credentials or large model outputs on-chain;
- rent bare GPUs or expose raw accelerator devices;
- accept arbitrary consumer-supplied containers, programs or models;
- expose a public shell, Docker socket or unrestricted host interface;
- put blockchain consensus in a hard real-time or safety-control loop;
- expose raw CAN, GPIO, serial, fieldbus or actuator access as a public service;
- reward a terminal merely for remaining online or declaring TOPS;
- hard-code one model provider, oracle, verifier or proof system;

- make third-party indexers authoritative for funds or permissions;
- give an agent runtime unrestricted owner custody by default;
- bundle unrelated execution engines into the production node.

17 Conclusion

TOS Core provides the implemented blockchain and networking substrate for persistent identity, bounded authority, asynchronous coordination and settlement. TOS Network is the broader product objective: an open service economy in which independently owned agents, models and physical terminals can transact without transferring custody of their hardware, data or operational policy to one platform.

The differentiation is not ownership of a particular accelerator or foundation model. It is the ability to determine which declared service may execute a request, under what limits, privacy and evidence policy, at what price, and how the result is receipted and settled. Physical terminals extend that model into real-world sites while preserving offline operation, deterministic local priority and independent safety authority.

The result is a focused positioning: TOS Network is being built as the open coordination and settlement network for AI services and physical edge intelligence. The chain foundation exists; the interoperable service protocol and terminal products remain work to be specified, implemented, tested and independently reviewed.

Copyright and License

Copyright © 2025–2026 TOS Network.

The source code and documentation in the TOS repository are distributed under the GNU General Public License v3.0. Protocol behavior is defined by released code, schemas and accepted network configuration; this whitepaper describes the architecture and direction and is not a substitute for implementation-level review. It is not an offer of securities or a promise of investment return, service adoption, protocol revenue or token appreciation.